🚀 Lecture 04.09: Dictionaries & Hash Table Architecture

📌 Executive Architectural Summary

A Python dictionary (`dict`) is an **associative mapping of unique, hashable keys to arbitrary values**. Since Python 3.6 (PEP 468) and guaranteed in 3.7+, dictionaries preserve **insertion order** while maintaining $\mathcal{O}(1)$ average-case lookup, insertion, and deletion complexity.

🔬 Core Engineering Mechanics

1. Hash Table Mapping

When inserting `d[k] = v` :

1. The hash function computes $h = exthash(k)$.
2. The index $i = h \pmod{2^k}$ determines the slot in the sparse index array.
3. The dense entries table stores `(hash, key, value)` .

2. Key Invariant: Hashability

A dictionary key **MUST be immutable and hashable** (possess a lifetime-constant `__hash__()` and `__eq__()`). Mutable objects (`list` , `set` , `dict`) cannot be used as keys (`TypeError: unhashable type`).

💻 Production Python Implementation

```
# 1. Dictionary Instantiation
student_marks = {
    "Shradha": 95,
    "Aman": 88,
    "Rohan": 76
}

# 2. O(1) Access and In-Place Mutation
print("Shradha's Marks:", student_marks["Shradha"])
student_marks["Priya"] = 92 # Insertion
student_marks["Aman"] = 90  # In-place update
print("Updated Dictionary:", student_marks)
```

```
# 3. Safe Lookup via .get()
score = student_marks.get("UnknownStudent", 0) # Defaults to 0, no KeyError
print("Safe Lookup Score:", score)
```